

COMP 421: Files & Databases

Lecture 11: B+Tree Optimizations and Concurrency

Announcements

P1 Due Today!

P2 Released Tonight

Code reviews will be week of October 5th

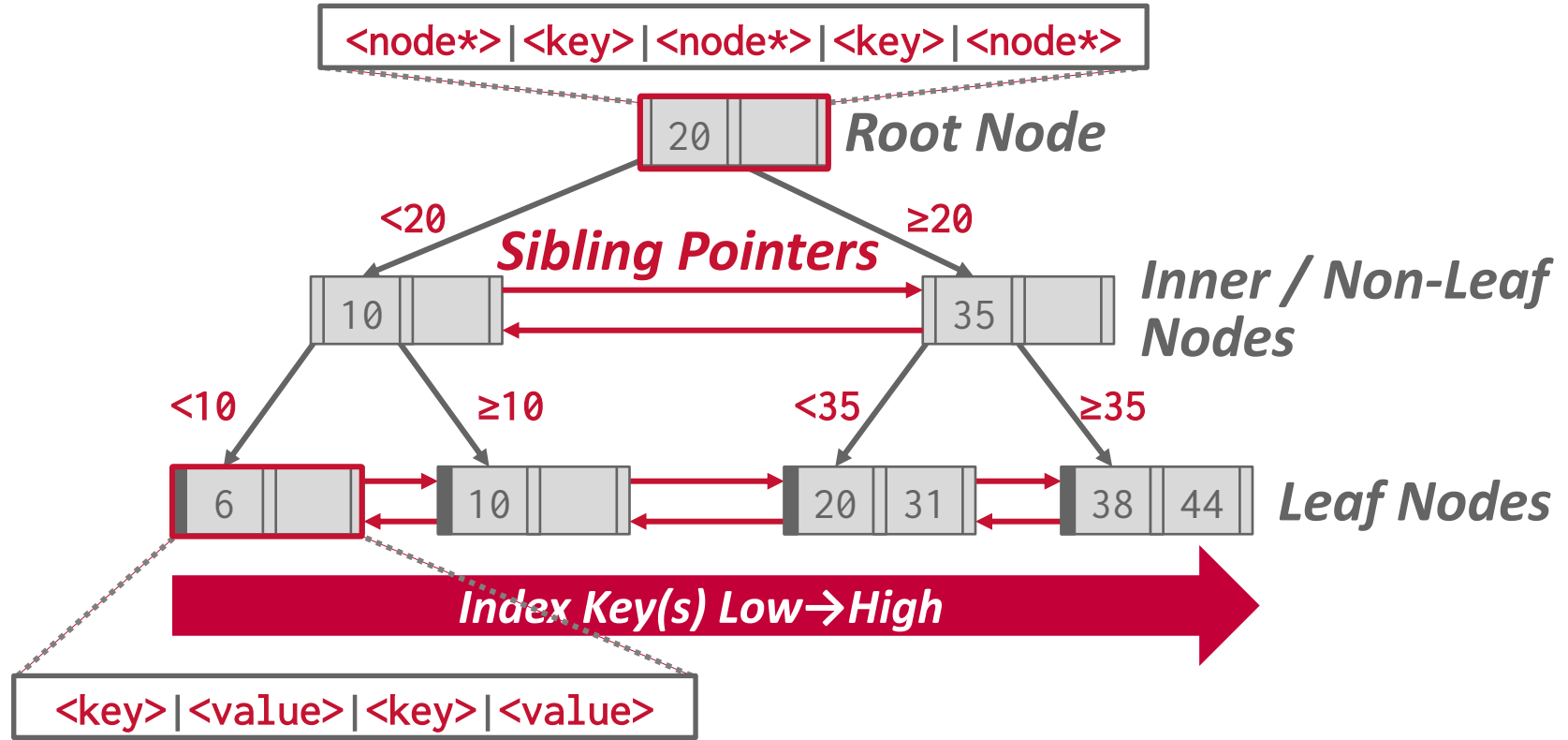
Last Class

A **B+Tree** is a self-balancing, ordered **m** -way tree for searches, sequential access, insertions, and deletions in **$O(\log_m n)$** where **m** is the tree fanout.

- It is perfectly balanced (i.e., every leaf node is at the same depth in the tree)
- Every node other than the root is at least half-full
 $m/2 - 1 \leq \#keys \leq m - 1$
- Every inner node with **k** keys has **$k + 1$** non-null children.
- Optimized for reading/writing large data blocks.

Some real-world implementations relax these properties, but we will ignore that for now...

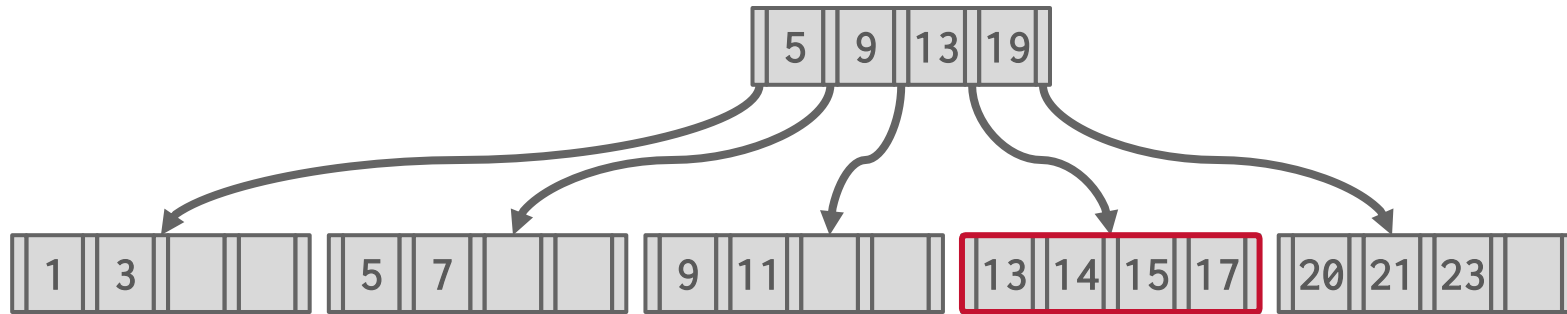
Last Class



B+TREE – INSERT EXAMPLE (3)

Insert 17

Insert 16

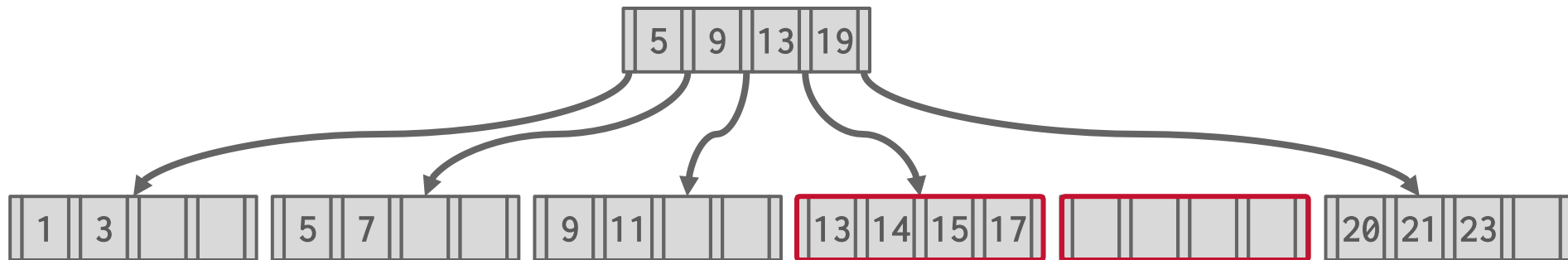


*Split the node
Copy the new key
'key' into the
new node.
Push the key up.*

B+TREE – INSERT EXAMPLE (3)

Insert 17

Insert 16



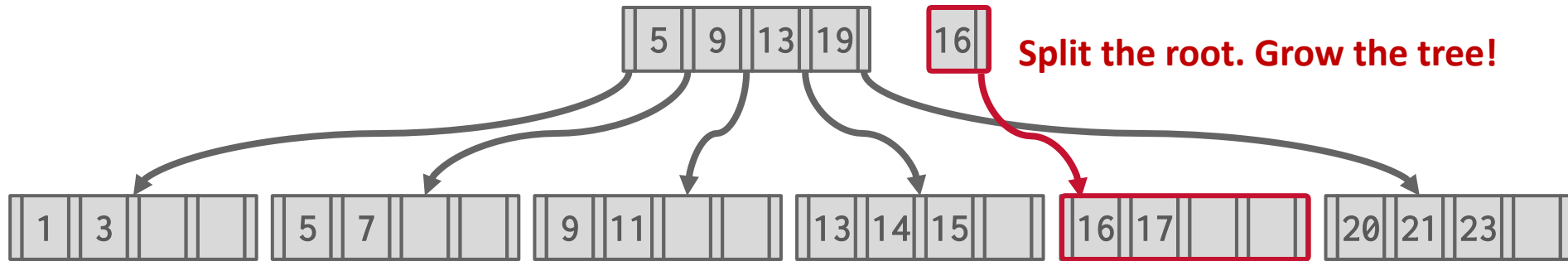
New Node!
***Shuffle keys from the
node that triggered the
split.***

B+TREE – INSERT EXAMPLE (3)

Insert 17

Insert 16

Want to create a key, pointer pair like this. But cannot insert it in the root node, which is full.

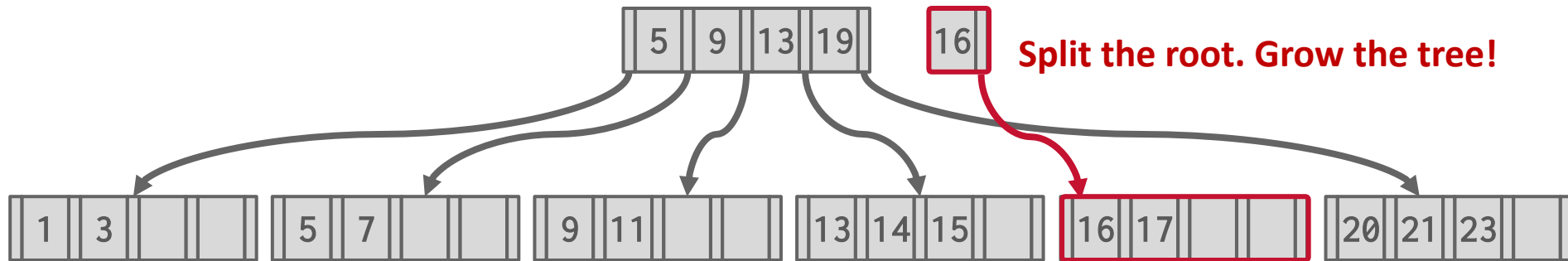


But this is an “orphan” node! No parent node points to it.

B+TREE – INSERT EXAMPLE (3)

Insert 17

Insert 16



B+TREE – INSERT EXAMPLE (3)

Insert 17

Insert 16



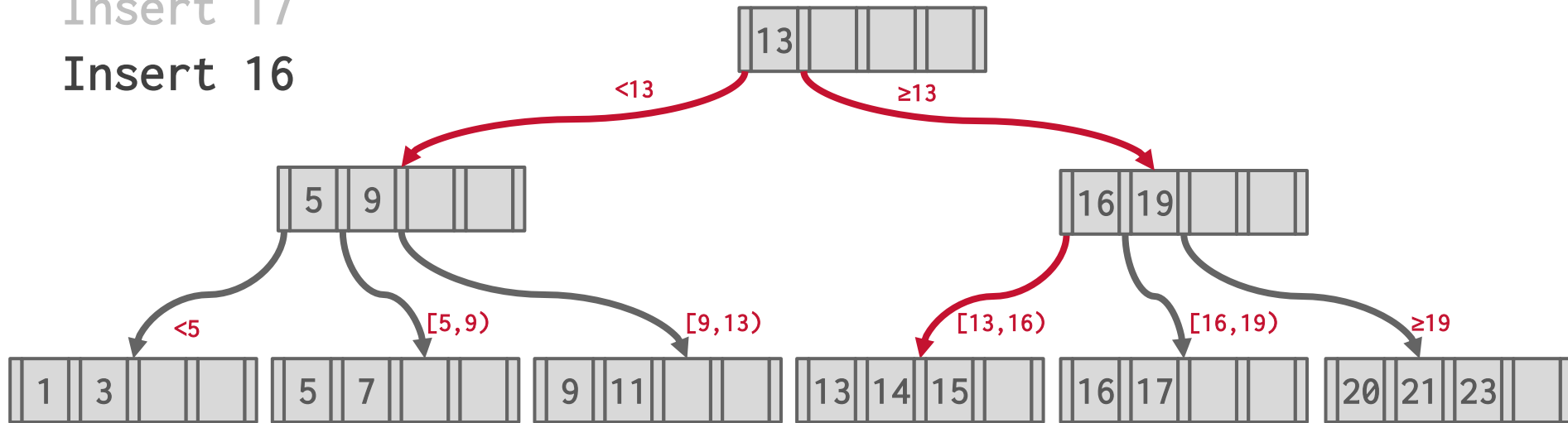
Next, need to split the “old” root, then point to the split nodes from the new root.



B+TREE – INSERT EXAMPLE (3)

Insert 17

Insert 16



B+Tree Optimizations, Design Choices

Locks, Latches, and Synchronization

B+Tree Concurrency

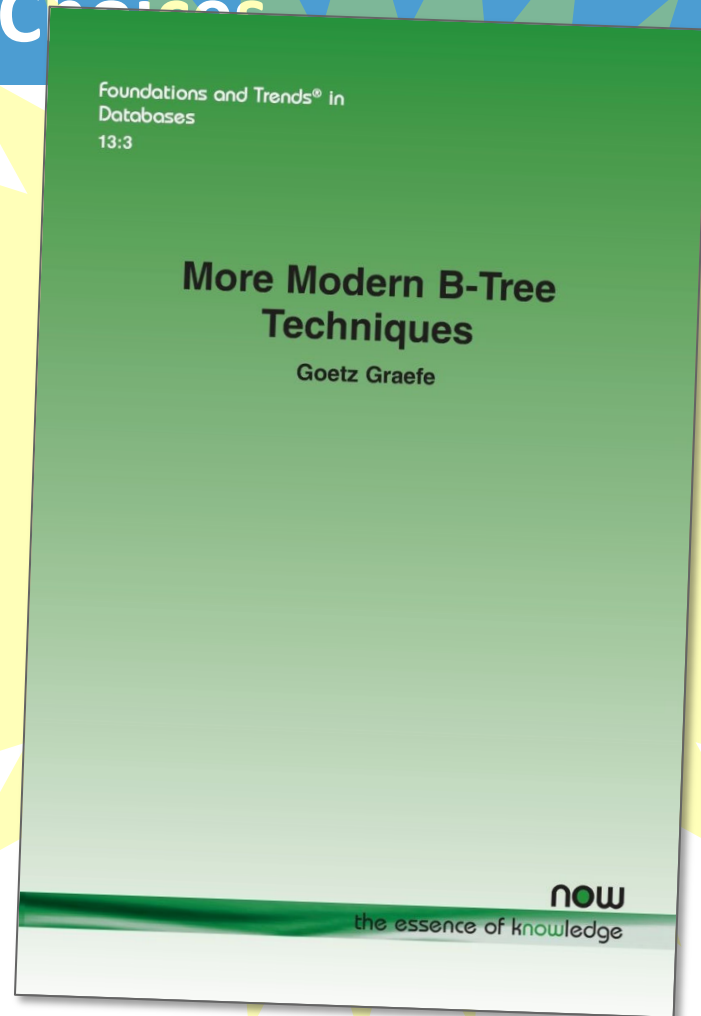
B+Tree Design Choices

Node Size

Merge Threshold

Variable-Length Keys

Intra-Node Search



Node Size

The slower the storage device, the larger the optimal node size for a B+Tree.

→ HDD: ~1MB

→ SSD: ~10KB

→ In-Memory: ~512B

Optimal sizes can vary depending on the workload

→ Leaf Node Scans vs. Root-to-Leaf Traversals

Merge Threshold

Some DBMSs do not always merge nodes when they are half full.

→ Average occupancy rate for B+Tree nodes is 69%.

Delaying a merge operation may reduce the amount of reorganization.

It may also be better to let smaller nodes exist and then periodically rebuild entire tree.

This is why PostgreSQL calls their B+Tree a "non-balanced" B+Tree ([nbtree](#)).

Variable-length Keys

Approach #1: Pointers

- Store the keys as pointers to the tuple's attribute.
- Also called T-Trees (in-memory DBMSs)

Approach #2: Variable-Length Nodes

- The size of each node in the index can vary.
- Requires careful memory management.

Approach #3: Padding

- Always pad the key to be max length of the key type.

Approach #4: Key Map / Indirection

- Embed an array of pointers that map to the key + value list within the node.

Intra-Node Search

Approach #1: Linear

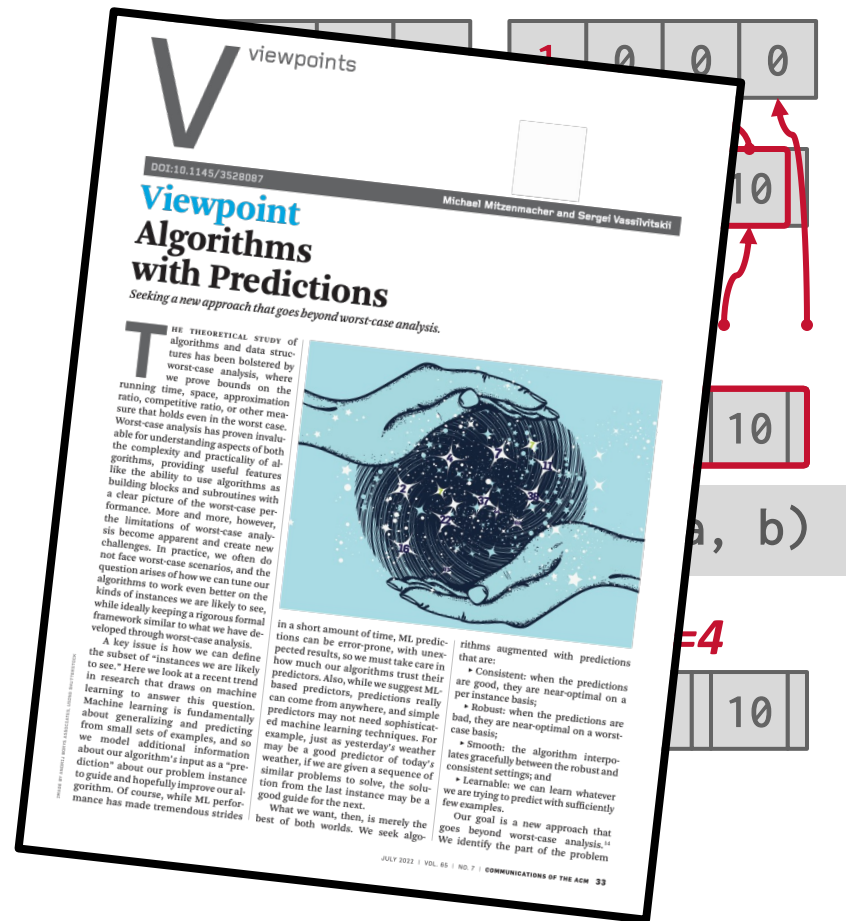
- Scan node keys from beginning to end.
- Use SIMD to vectorize comparisons.

Approach #2: Binary

- Jump to middle key, pivot left/right depending on comparison.

Approach #3: Interpolation

- Approximate location of desired key based on known distribution of keys.



Optimizations

Prefix Compression

Deduplication

Suffix Truncation

Pointer Swizzling

Bulk Insert

Buffered Updates

Many more...

Prefix Compression

Sorted keys in the same leaf node are likely to have the same prefix.

Instead of storing the entire key each time, extract common prefix and store only unique suffix for each key.

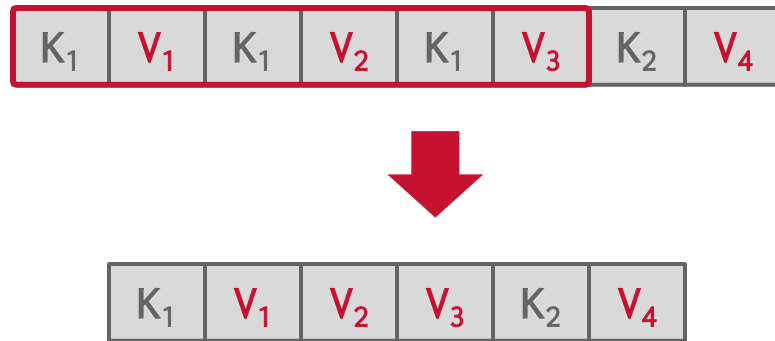
→ Many variations.



Deduplication

Non-unique indexes can end up storing multiple copies of the same key in leaf nodes.

The leaf node can store the key once and then maintain a "posting list" of tuples with that key.

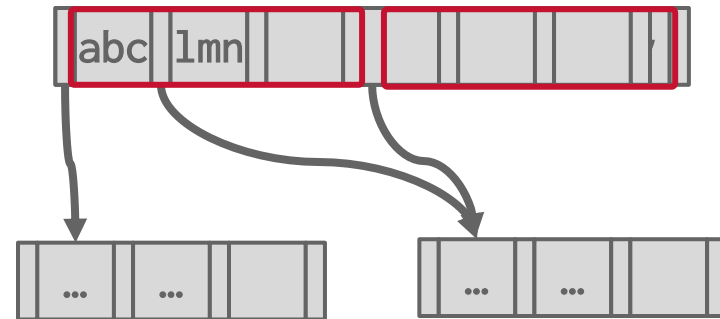


Suffix Truncation

The keys in the inner nodes are only used to "direct traffic".

→ We don't need the entire key.

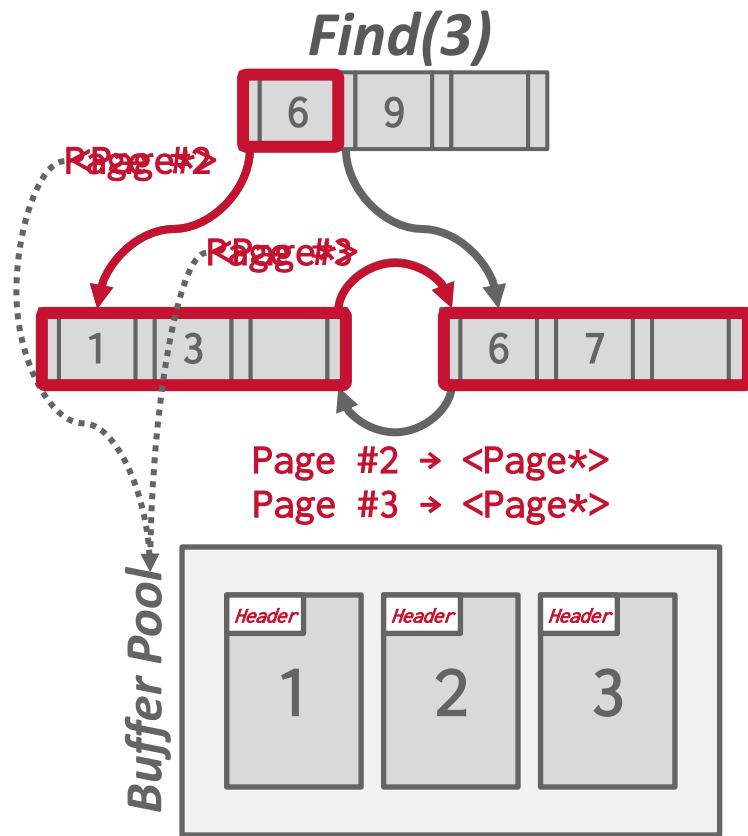
Store a minimum prefix that is needed to correctly route probes into the index.



Pointer Swizzling

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.

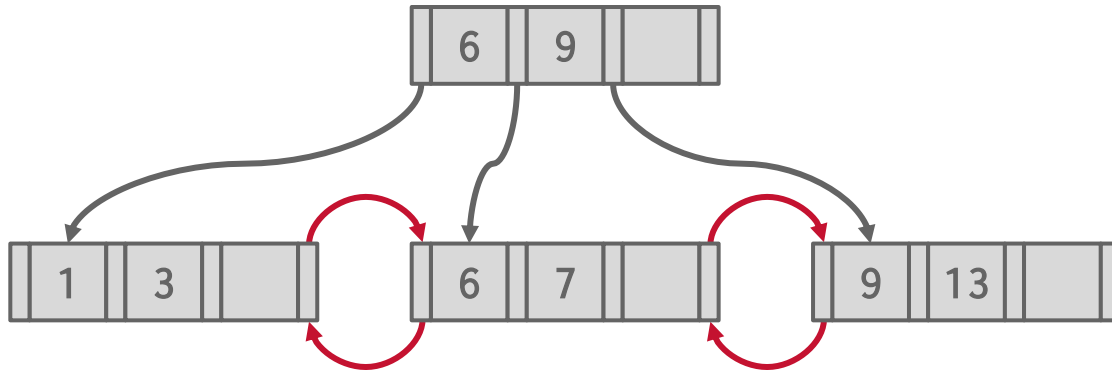


Bulk Insert

The fastest way to build a new B+Tree for an existing table is to first sort the keys and then build the index from the bottom up.

Keys: 3, 7, 9, 13, 6, 1

Sorted Keys: 1, 3, 6, 7, 9, 13



Observation

Modifying a B+tree is expensive when the DBMS has to split/merge nodes.

- Worst case is when DBMS reorganizes the entire tree.
- The worker that causes a split/merge is responsible for doing the work.

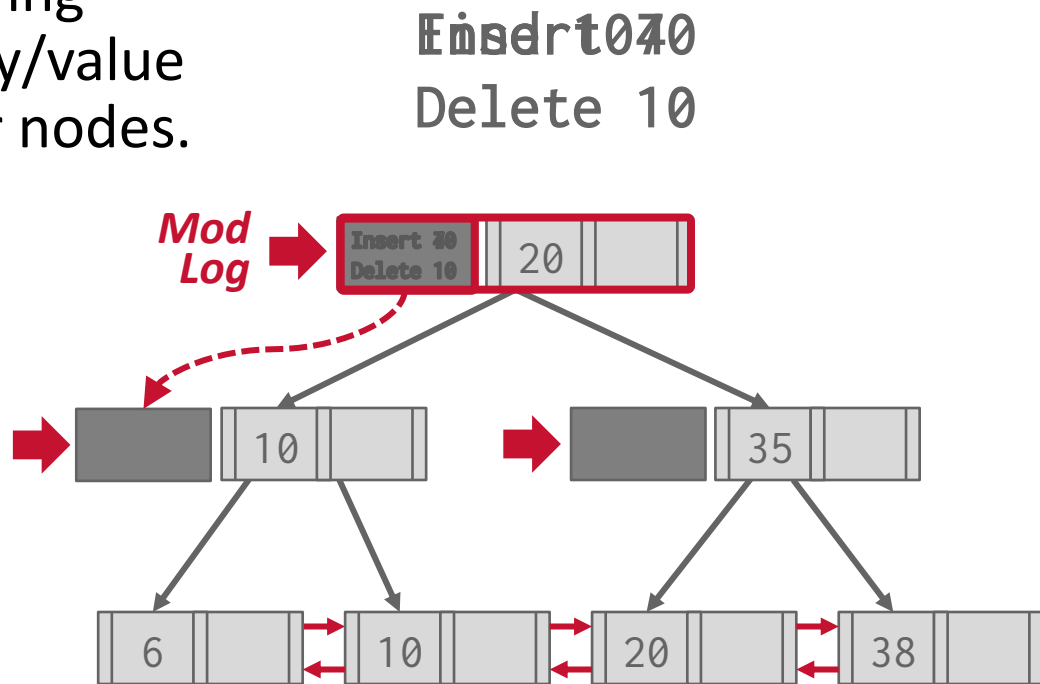
What if there was a way to delay updates and then apply multiple changes together in a batch?

Write-Optimized B+Tree (B ϵ -Tree)

Instead of immediately applying updates, store changes to key/value entries in log buffers at inner nodes.

→ aka **Fractal Trees** / **B ϵ -trees**.

Updates cascade down to lower nodes incrementally when buffers get full.



Tokute

DB

Rela

moDB

STS



UNC
DEPARTMENT OF
COMPUTER SCIENCE

Observation

So far, we have given thread *unsafe* B+Tree algorithms

A modern DBMS needs to allow multiple threads to safely access data structures to take advantage of additional CPU cores and hide disk I/O stalls.



They Don't Do This!

VOLTDB **KX**

Redis **H-Store**



Concurrency Control

A **concurrency control** protocol is the method that the DBMS uses to ensure “correct” results for concurrent operations on a shared object.

A protocol's correctness criteria can vary:

- **Logical Correctness:** Can a thread see the data that it is supposed to see?
- **Physical Correctness:** Is the internal representation of the object sound?

B+ Tree Concurrency Control

Latches Overview

B+Tree Latching

Leaf Node Scans

Locks vs. Latches

Locks (Transactions)

- Protect the database's logical contents from other transactions.
- Held for transaction's duration.
- Need to be able to rollback changes.

Latches (Workers)

- Protect the critical sections of the DBMS's internal data structure from other workers (e.g., threads).
- Held for operation duration.
- Do not need to be able to rollback changes.

Locks vs. Latches

Lecture #15



Locks

Separate...	Transactions
Protect...	Database Contents
During...	Entire Transactions
Modes...	Shared, Exclusive, Update, Intention
Deadlock	Detection & Resolution
...by...	Waits-for, Timeout, Aborts
Kept in...	Lock Manager

Latches

Workers (threads, processes)
In-Memory Data Structures
Critical Sections
Read, Write
Avoidance
Coding Discipline
Protected Data Structure

Source: [Goetz Graefe](#)

Latch Implementation Goals

Small memory

Fast execution

Decentralized

I repeat: **do not use spinlocks in user space, unless you actually know what you're doing.** And be aware that the likelihood that you know what you are doing is basically nil.

By: **Linus Torvalds** (torvalds.delete@this.linux-foundation.org), January 3, 2020 6:05 pm

Room: Moderated Discussions

Beastian (no.email.delete@this.aol.com) on January 3, 2020 11:46 am wrote:
> I'm usually on the other side of these primitives when I write code as a consumer of them,
> but it's very interesting to read about the nuances related to their implementations:

The whole post seems to be just wrong, and is measuring something completely different than what the author thinks and claims it is measuring.

First off, spinlocks can only be used if you actually know you're not being scheduled while using them. But the blog post author seems to be implementing his own spinlocks in user space with no regard for whether the lock user might be scheduled or not. And the code used for the claimed "lock not held" timing is complete garbage.

It basically reads the time before releasing the lock, and then it reads it after acquiring the lock again, and claims that the time difference is the time when no lock was held. Which is just inane and pointless and completely wrong.

That's pure garbage. What happens is that

- (a) since you're spinning, you're using CPU time
- (b) at a random time, the scheduler will schedule you out
- (c) that random time might be just after you acquire the lock

And then you write a blog-post blaming others, not understanding that it's your incorrect code that is garbage, and is giving random garbage values.

Latch Implementations

Atomic Instructions (Non-Blocking) OS Mutex Reader-Writer Locks

Advanced approach



WebKit

Locking in WebKit

May 6, 2016 by Filip Pizlo @filpizlo

Back in August 2015 we replaced all `spinlocks` and `OS-provided mutexes` in WebKit with the new `WTF::Lock` (WTF stands for Web Template Framework). We also replaced all OS-provided condition variables with `WTF::Condition`. These new primitives have some cool properties:

1. `WTF::Lock` and `WTF::Condition` only require one byte of storage each. `WTF::Lock` only needs two bits in that byte. The small size encourages using huge numbers of very fine-grained locks. OS mutexes often require 64 bytes or more. The small size of `WTF::Lock` means that there's rarely an excuse for not having one, or even multiple, fine-grained locks in any object that has things that need to be synchronized.

Compared to OS-provided locks like `pthread_mutex`, `WTF::Lock` is 64 times smaller and up to 180 times faster. Compared to OS-provided condition variables like `pthread_cond`, `WTF::Condition` is 64 times smaller. Using `WTF::Lock` instead of `pthread_mutex` means that WebKit is 10% faster on [JetStream](#), 5% faster on [Speedometer](#), and 5% faster on our page loading test.

4. `WTF::Lock` doesn't waste CPU cycles when a lock is held for a long time. `WTF::Lock` is adaptive: it changes its strategy for how to wait for the lock to become available based on how long it has been trying. If the lock doesn't become available promptly, `WTF::Lock` will suspend the calling thread until the lock becomes available.



Latch Implementations

Approach #1: Test-and-Set Spin Latch (TAS)

- Very efficient (single instruction to latch/unlatch)
- Non-scalable, not cache friendly, not OS friendly.
- Example: `std::atomic_flag`

Not `std::atomic<bool>`

```
std::atomic_flag latch;  
:  
while (latch.test_and_set()) {  
    // Retry? Yield? Abort?  
}
```

test_and_set semantics:
-if flag is 1, return **TRUE**
-if flag is 0, flip it to 1 and return **FALSE**
-executes **atomically**, no **mutex**

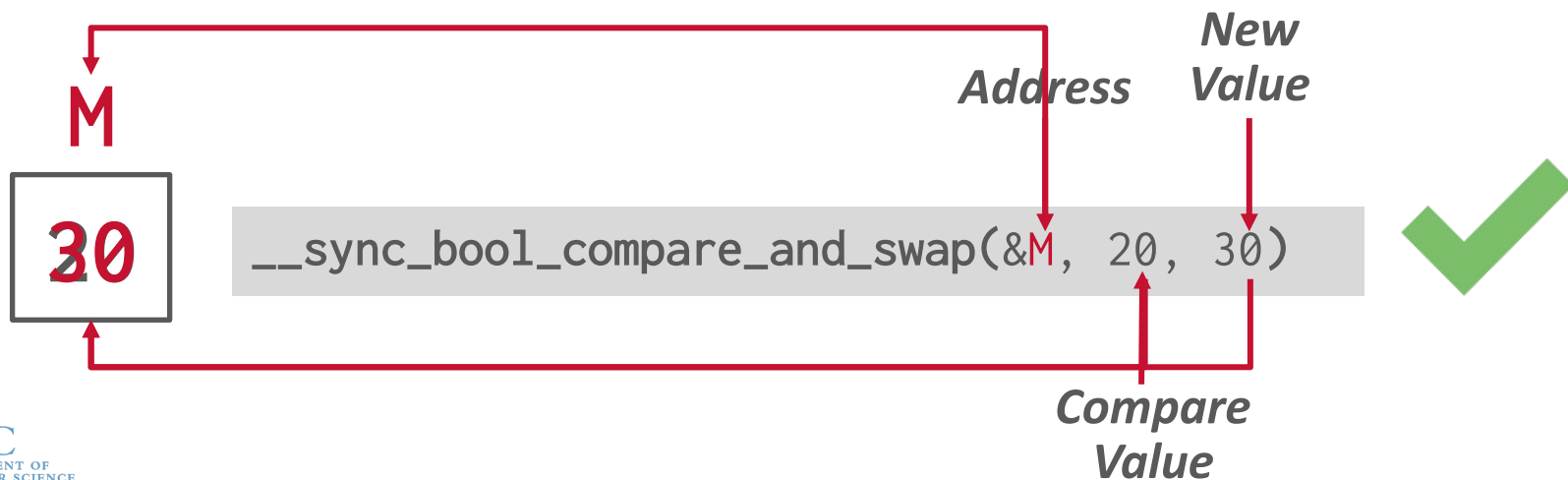
Compare-and-swap (CAS)

Atomic instruction that compares contents of a memory location **M** to a given value **V**

→ If values are equal, installs new given value **V'** in **M**, **return TRUE**

→ Otherwise, operation fails, **return FALSE**

See [C++11 Atomics](#)



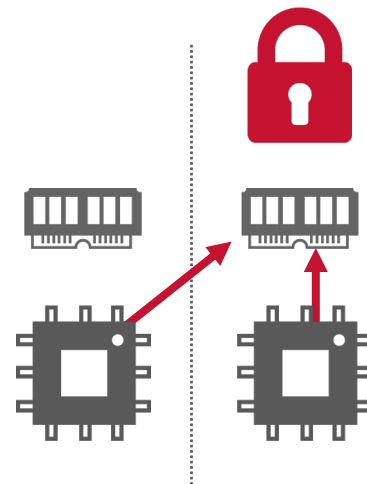
Latch Implementations

Approach #1: Test-and-Set Spin Latch (TAS)

- Very efficient (single instruction to latch/unlatch)
- Non-scalable, not cache friendly, not OS friendly.
- Example: `std::atomic_flag`

Not `std::atomic<bool>`

```
std::atomic_flag latch;  
:  
while (latch.test_and_set()) {  
    // Retry? Yield? Abort?  
}
```

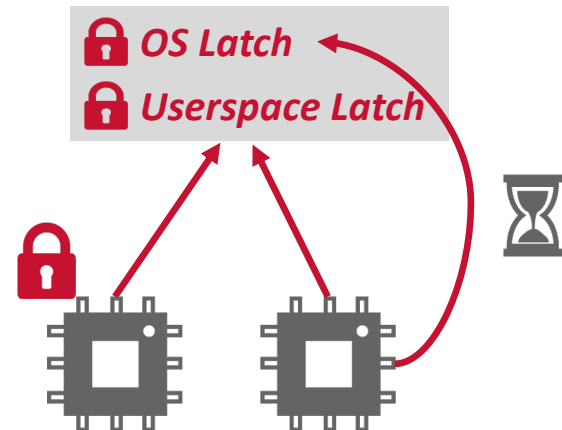


Latch Implementations

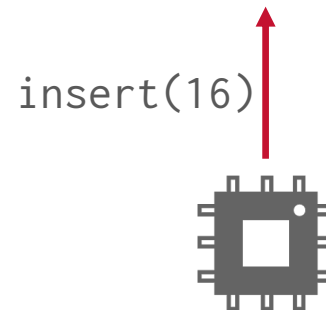
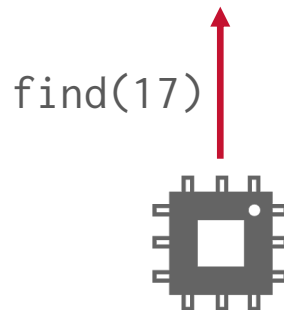
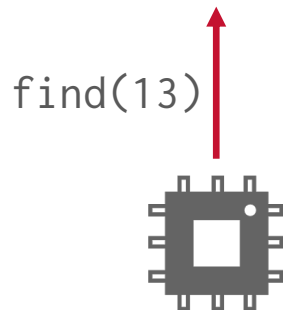
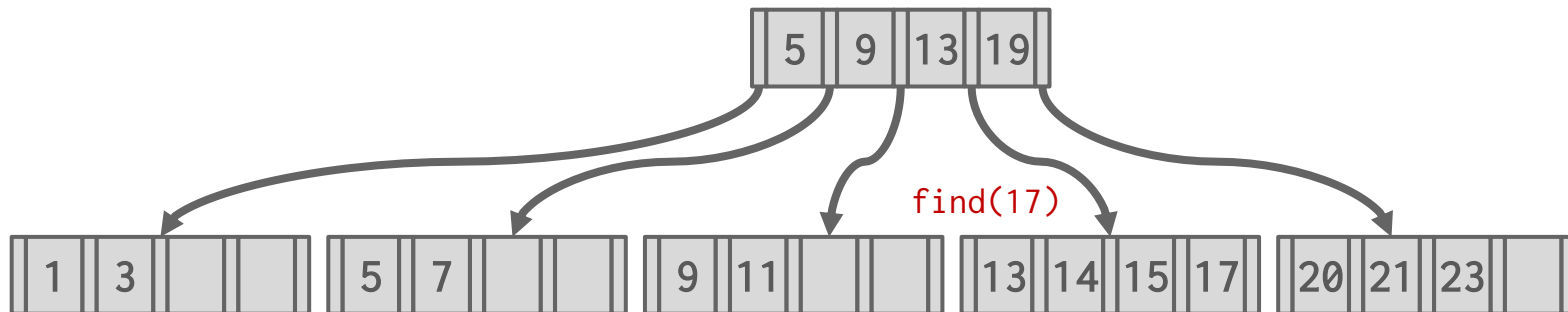
Approach #2: Blocking OS Mutex

- Simple to use
- Non-scalable (about 25ns per lock/unlock invocation)
- Example: `std::mutex` → `pthread_mutex_t` → `futex`

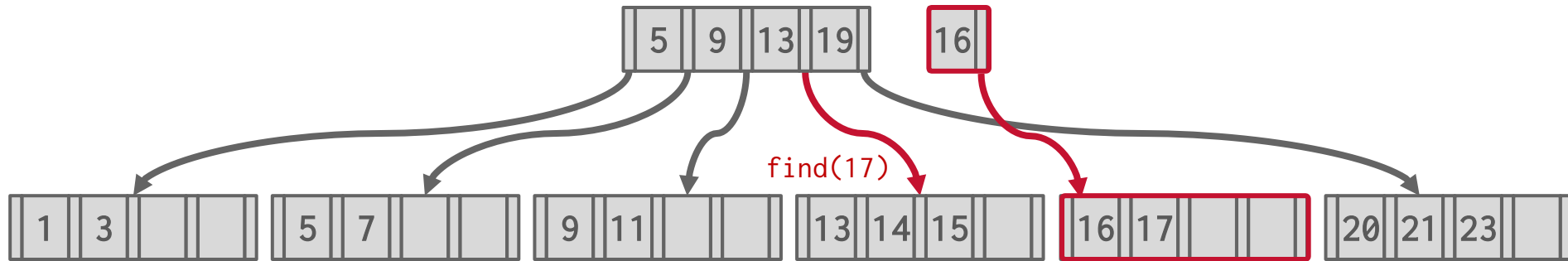
```
std::mutex m;  
:  
m.lock();  
// Do something special...  
m.unlock();
```



Observation



Observation



Idea: Readers can share access with other readers.
Writers can't share access with anyone (readers or writers).

Latch Modes

Read Mode

- Multiple threads can read the same object at the same time.
- A thread can acquire the read latch if another thread has it in read mode.

Write Mode

- Only one thread can access the object.
- A thread cannot acquire a write latch if another thread has it in any mode.

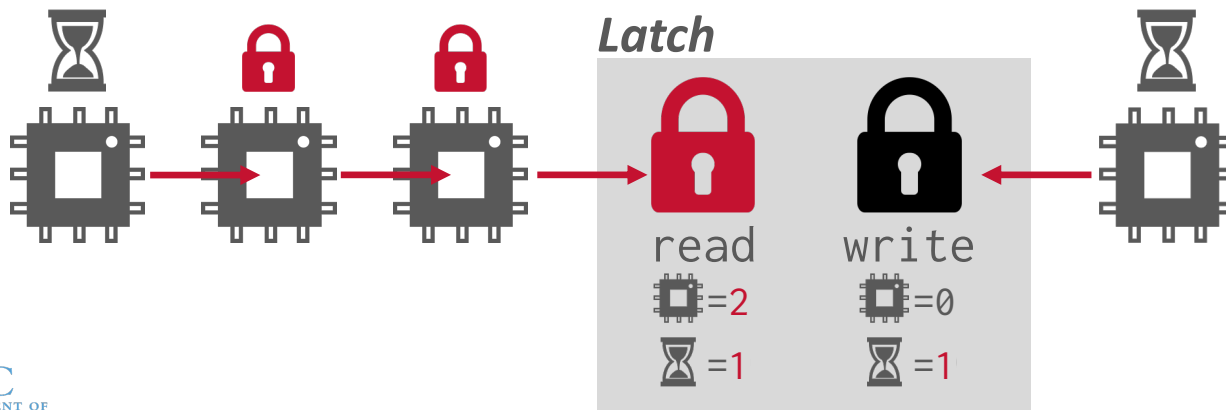
Compatibility Matrix

	Read	Write
Read	✓	X
Write	X	X

Latch Implementations

Approach #3: Reader-Writer Latches

- Allows for concurrent readers. Must manage read/write queues to avoid starvation.
- Can be implemented on top of spinlocks. → pthread_mutex_t
→ pthread_cond_t
- Example: `std::shared_mutex` → pthread_rwlock_t



B+Tree Concurrency Control

We want to allow multiple threads to read and update a B+Tree at the same time.

We need to protect against two types of problems:

- Threads trying to modify the contents of a node at the same time.
- One thread traversing the tree while another thread splits/merges nodes.

Latch Crabbing/Coupling

Protocol to allow multiple threads to access/modify B+Tree at the same time.

- Get latch for parent
- Get latch for child
- Release latch for parent if “safe”

A **safe node** is one that will not split or merge when updated.

- Not full (on insertion)
- More than half-full (on deletion)

Latch Crabbing/Coupling

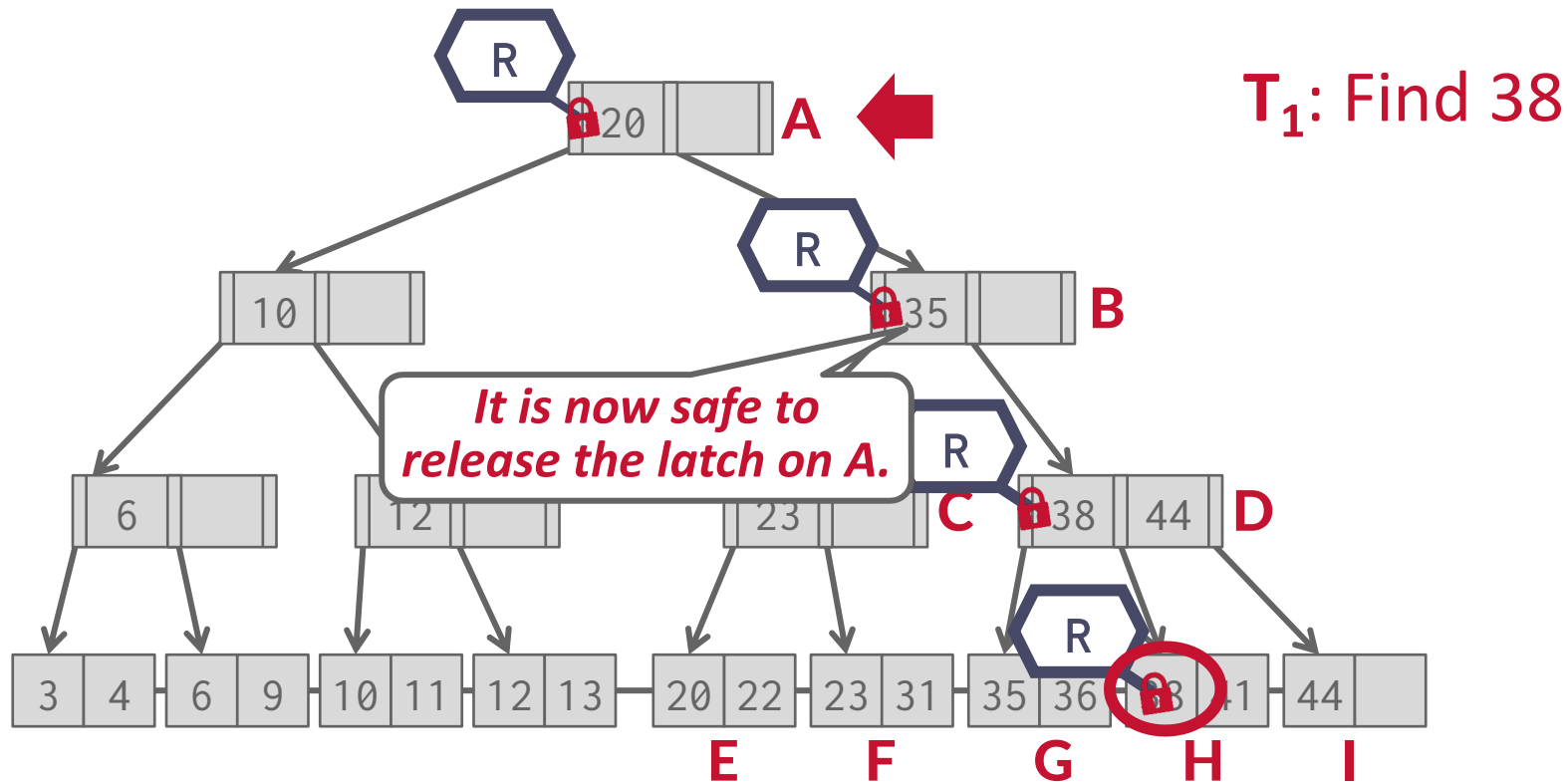
Find: Start at root and traverse down the tree:

- Acquire **R** latch on child,
- Then unlatch parent.
- Repeat until we reach the leaf node.

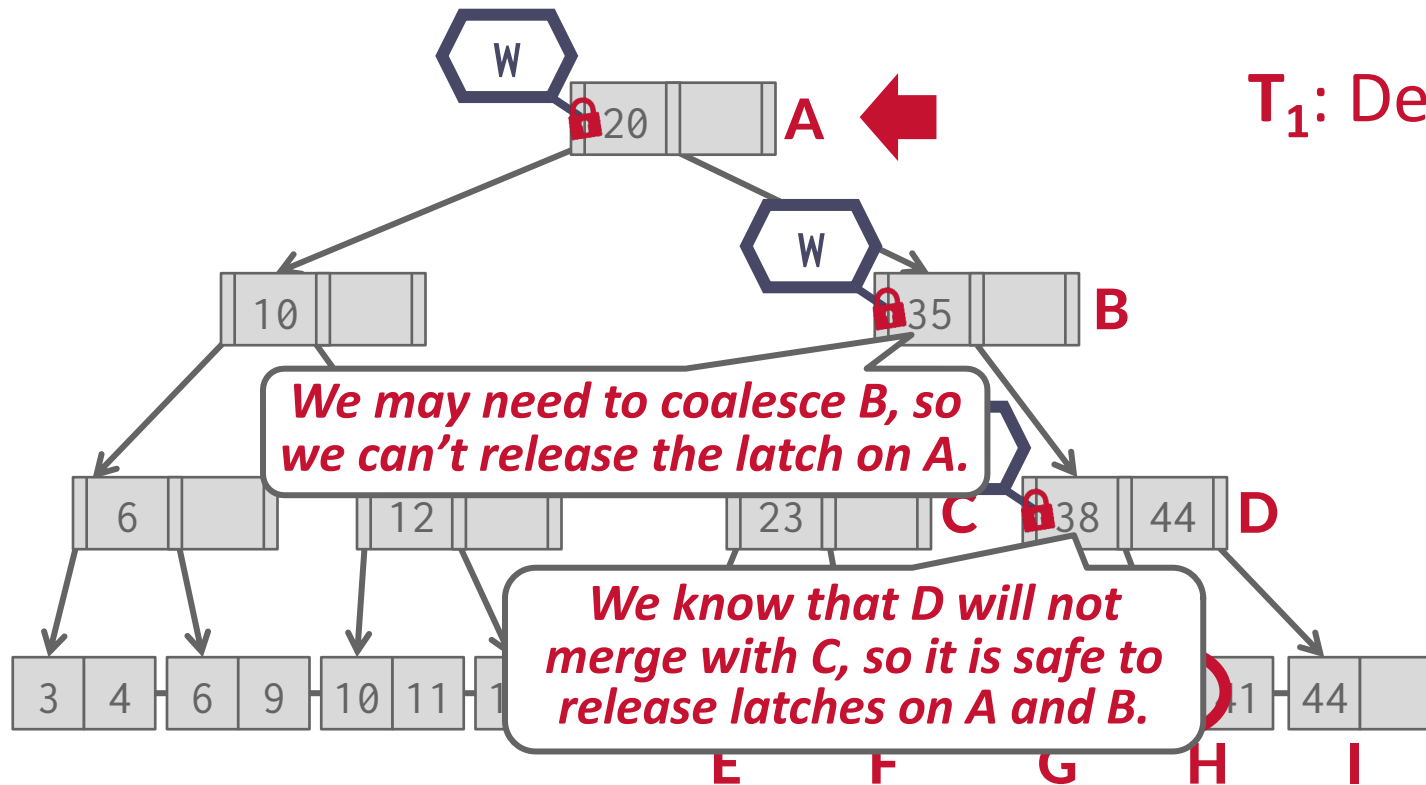
Insert/Delete: Start at root and go down, obtaining **W** latches as needed. Once child is latched, check if it is safe:

- If child is safe, release all latches on ancestors

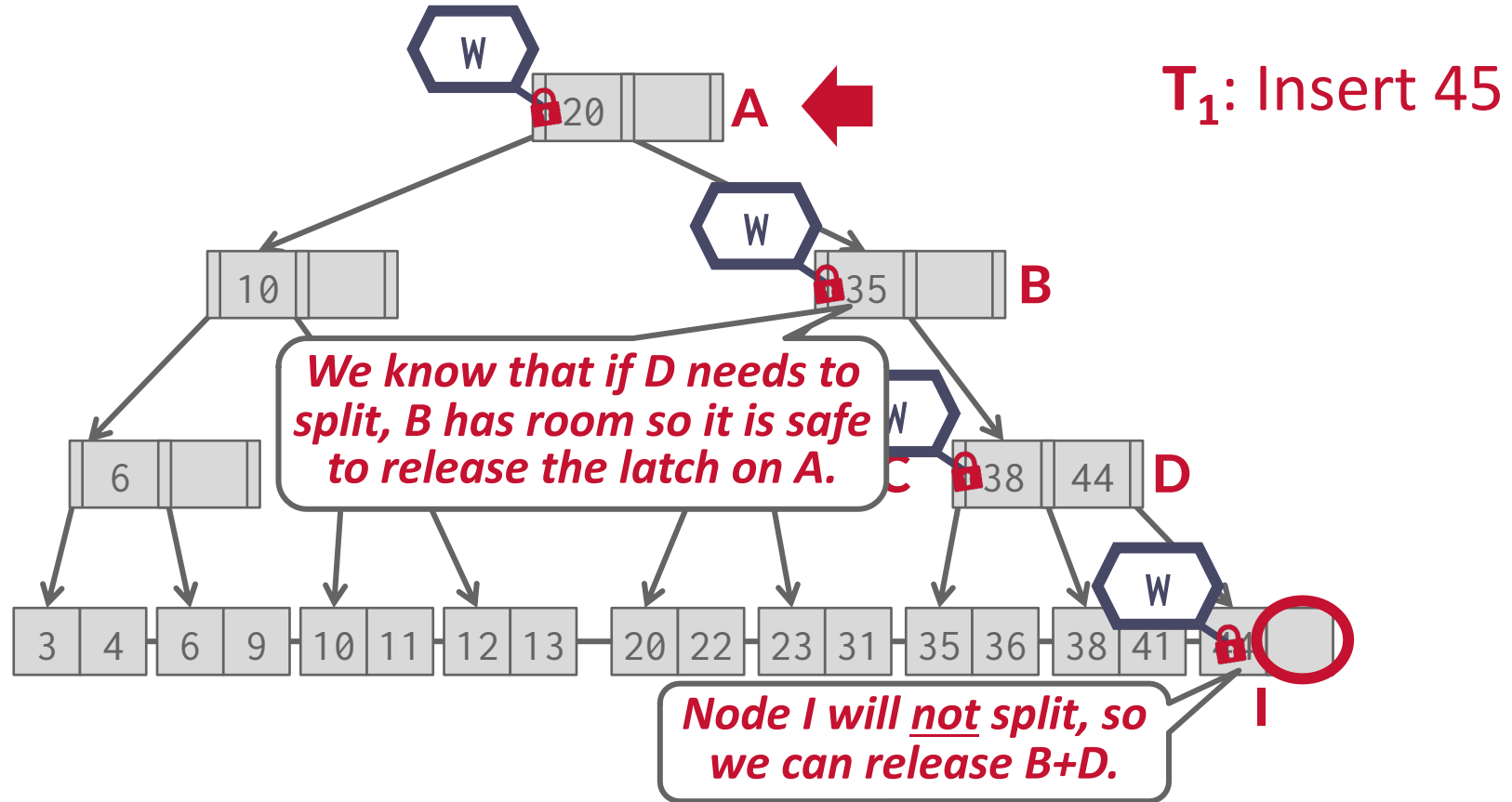
Example #1 – FIND 38



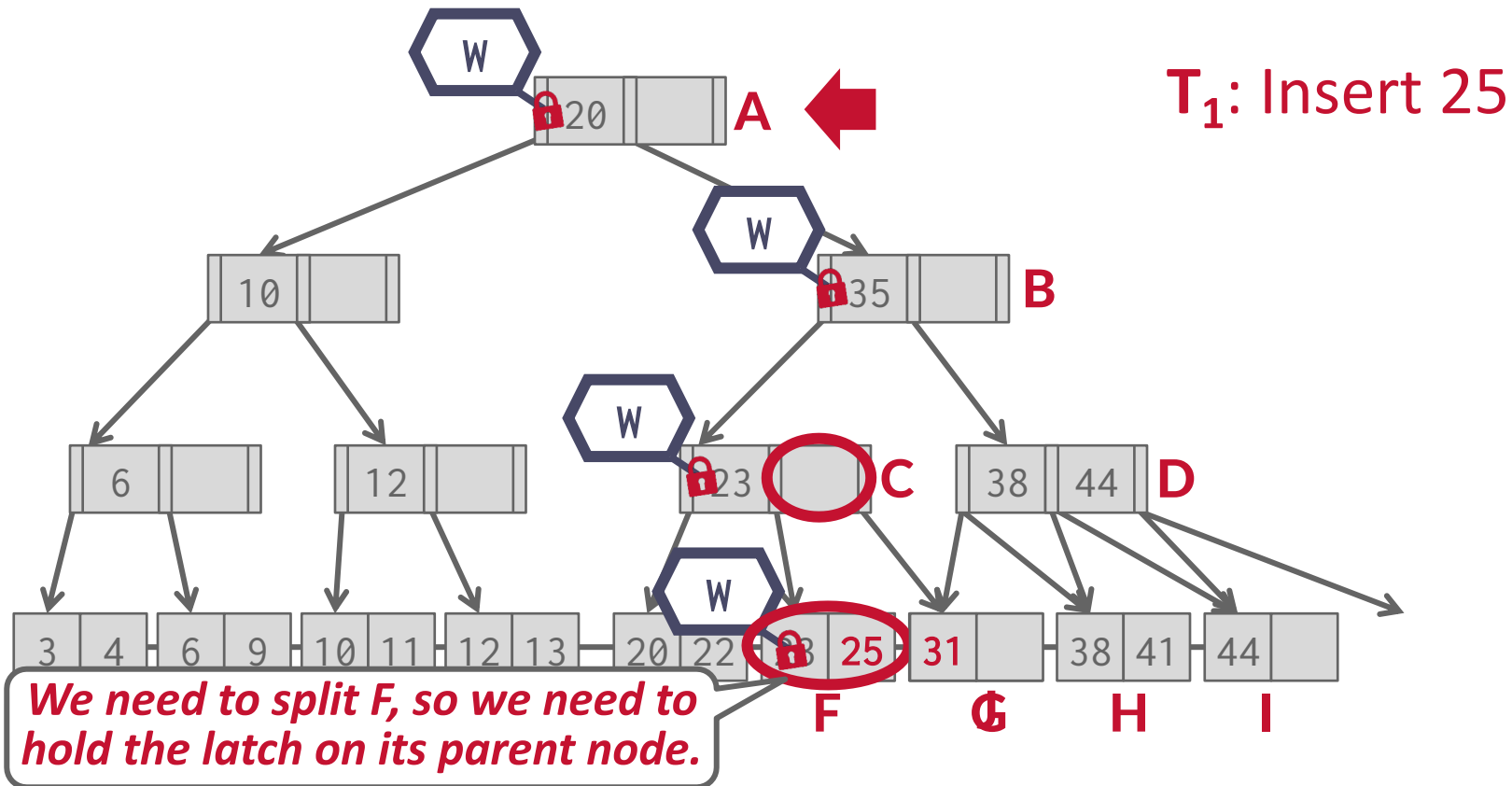
Example #2 – DELETE 38



Example #3 – INSERT 45

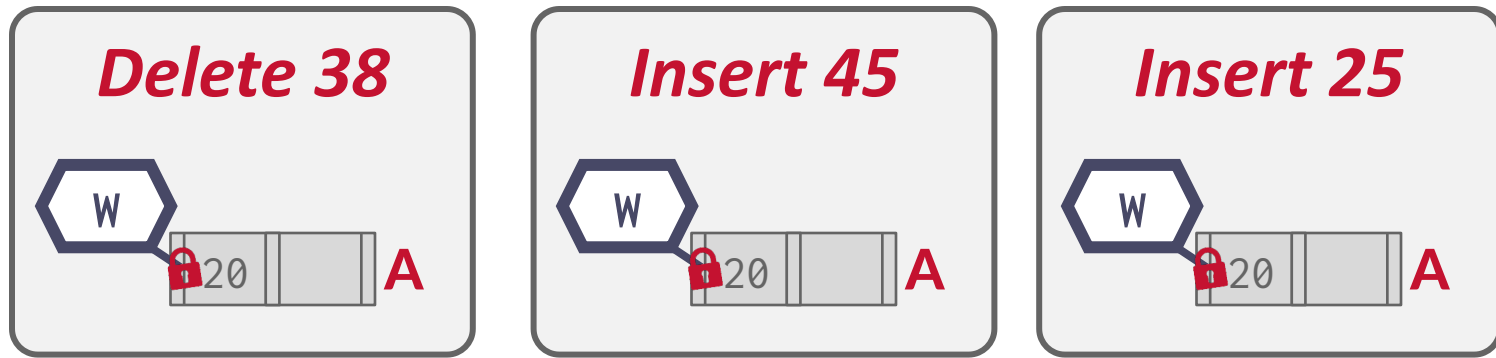


Example #4 – INSERT 25



Observation

What was the first step that all the update examples did on the B+Tree?



Taking a write latch on the root every time becomes a bottleneck with higher concurrency.

Better Latching Algorithm

Most modifications to a B+Tree will not require a split or merge.

Instead of assuming there will be a split/merge, optimistically traverse the tree using read latches.

If a worker guesses wrong, repeat traversal with pessimistic algorithm.

Acta Informatica 9, 1–21 (1977)



Concurrency of Operations on B-Trees

R. Bayer* and M. Schkolnick
IBM Research Laboratory, San José, CA 95193, USA

Summary. Concurrent operations on B-trees pose the problem of insuring that each operation can be carried out without interfering with other operations being performed simultaneously by other users. This problem can become critical if these structures are being used to support access paths, like indexes, to data base systems. In this case, serializing access to one of these indexes can create an unacceptable bottleneck for the entire system. Thus, there is a need for locking protocols that can assure integrity for each access while at the same time providing a maximum possible degree of concurrency. Another feature required from these protocols is that they be deadlock free, since the cost to resolve a deadlock may be high.

Recently, there has been some questioning on whether B-tree structures can support concurrent operations. In this paper, we examine the problem of concurrent access to B-trees. We present a deadlock free solution which can be tuned to specific requirements. An analysis is presented which allows the selection of parameters so as to satisfy these requirements.

The solution presented here uses simple locking protocols. Thus, we conclude that B-trees can be used advantageously in a multi-user environment.

1. Introduction

In this paper, we examine the problem of concurrent access to indexes which are maintained as B-trees. This type of organization was introduced by Bayer and McCreight [2] and some variants of it appear in Knuth [10] and Wedekind [13]. Performance studies of it were restricted to the single user environment. Recently, these structures have been examined for possible use in a multi-user (concurrent) environment. Some initial studies have been made about the feasibility of their use in this type of situation [1, 6], and [11].

An accessing schema which achieves a high degree of concurrency in using the index will be presented. The schema allows dynamic tuning to adapt its performance to the profile of the current set of users. Another property of the

* Permanent address: Institut für Informatik der Technischen Universität München, Arcisstr. 21, D-8000 München 2, Germany (Fed. Rep.)



Better Latching Algorithm

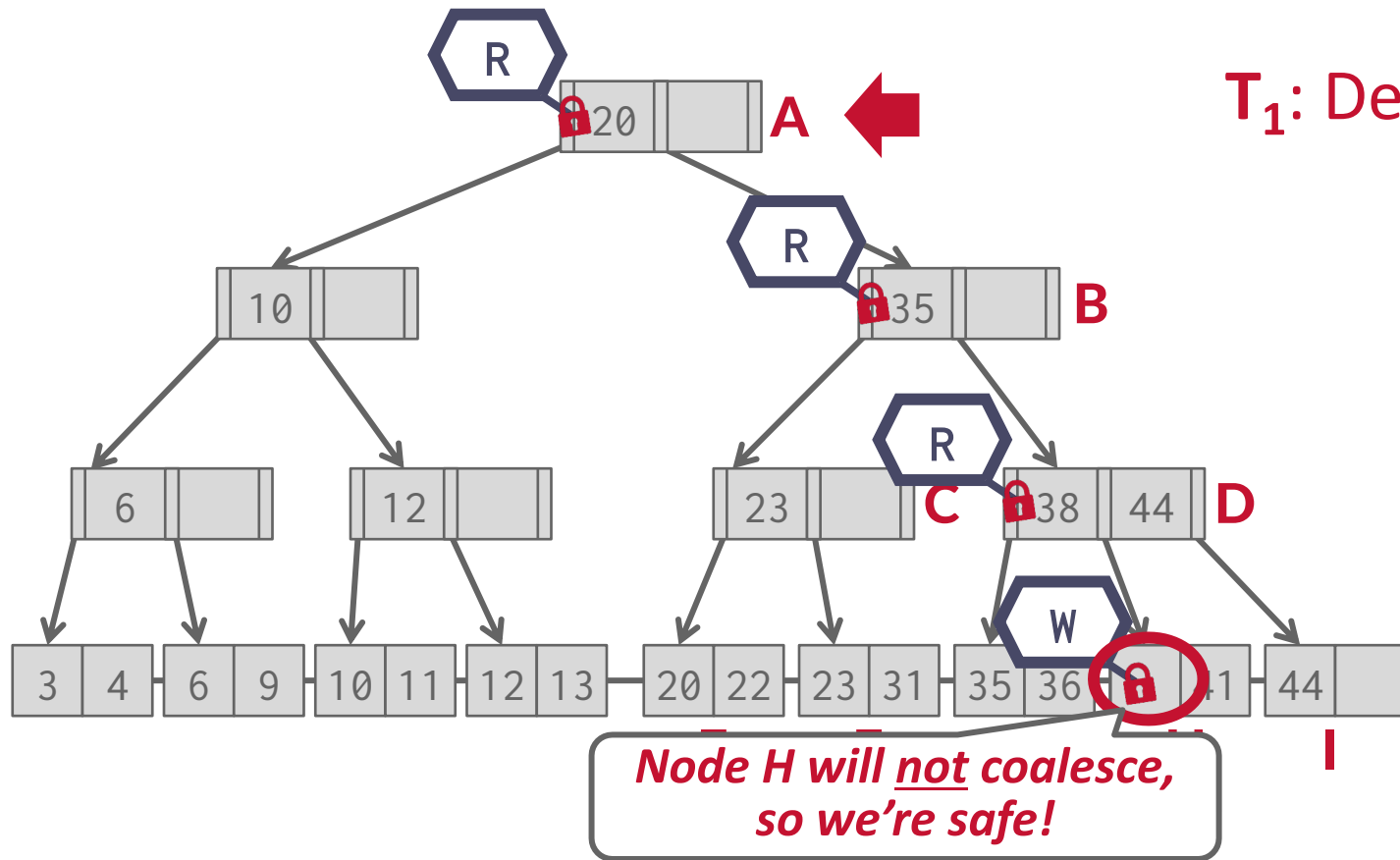
Search: Same as before.

Insert/Delete:

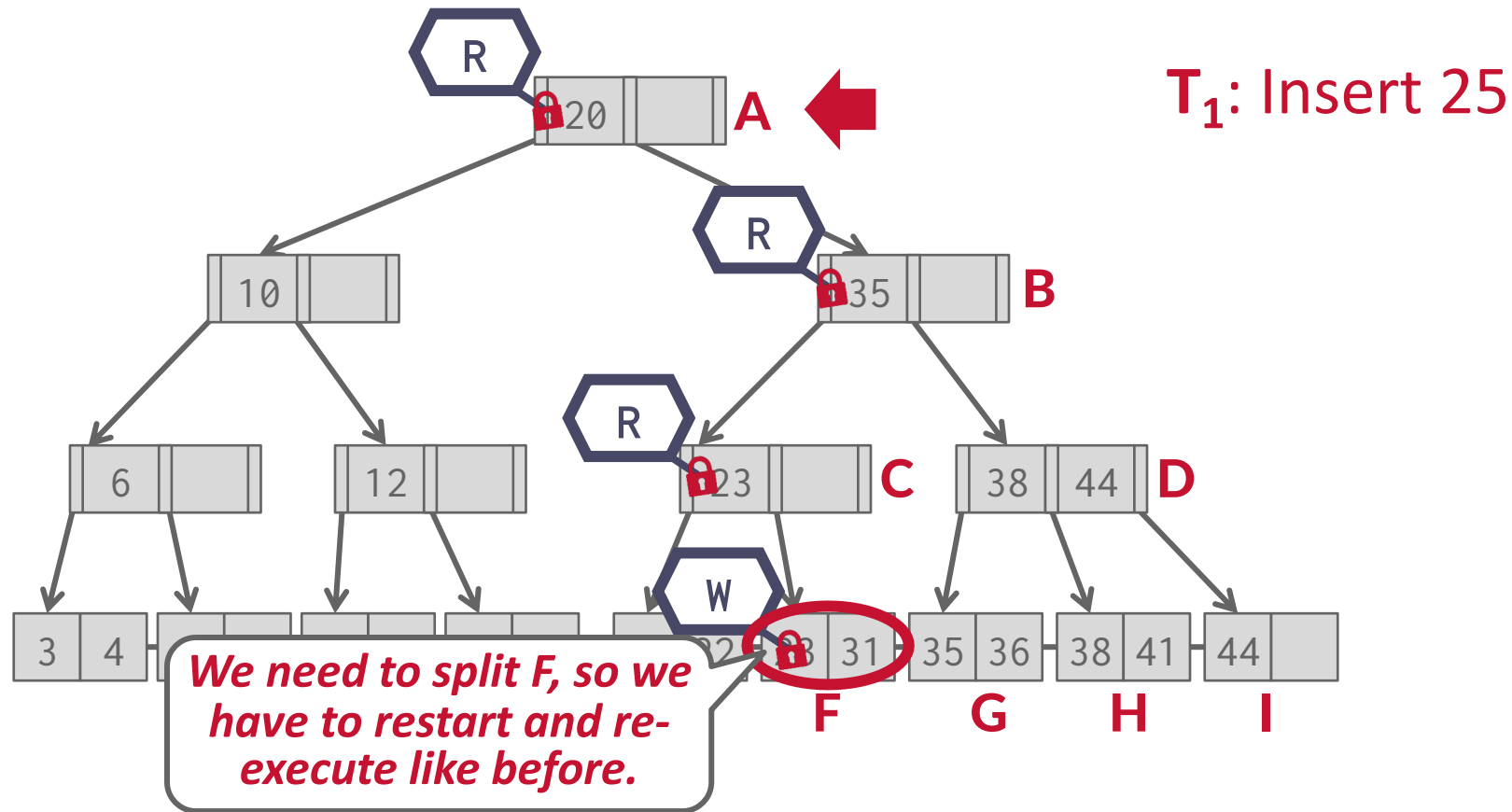
- Set latches as if for search, get to leaf, and set **W** latch on leaf.
- If leaf is not safe, release all latches, and restart thread using previous insert/delete protocol with write latches.

This approach optimistically assumes that only leaf node will be modified; if not, **R** latches set on the first pass to leaf are wasteful.

Example #2 – DELETE 38



Example #4 – INSERT 25



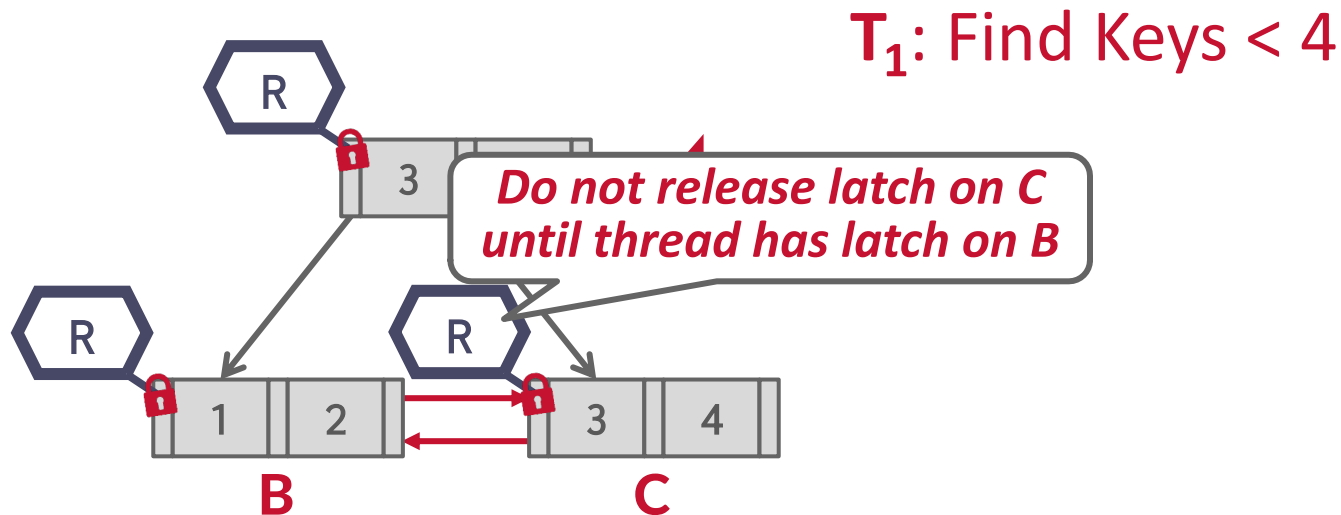
Observation

The threads in all the examples so far have acquired latches in a "top-down" manner.

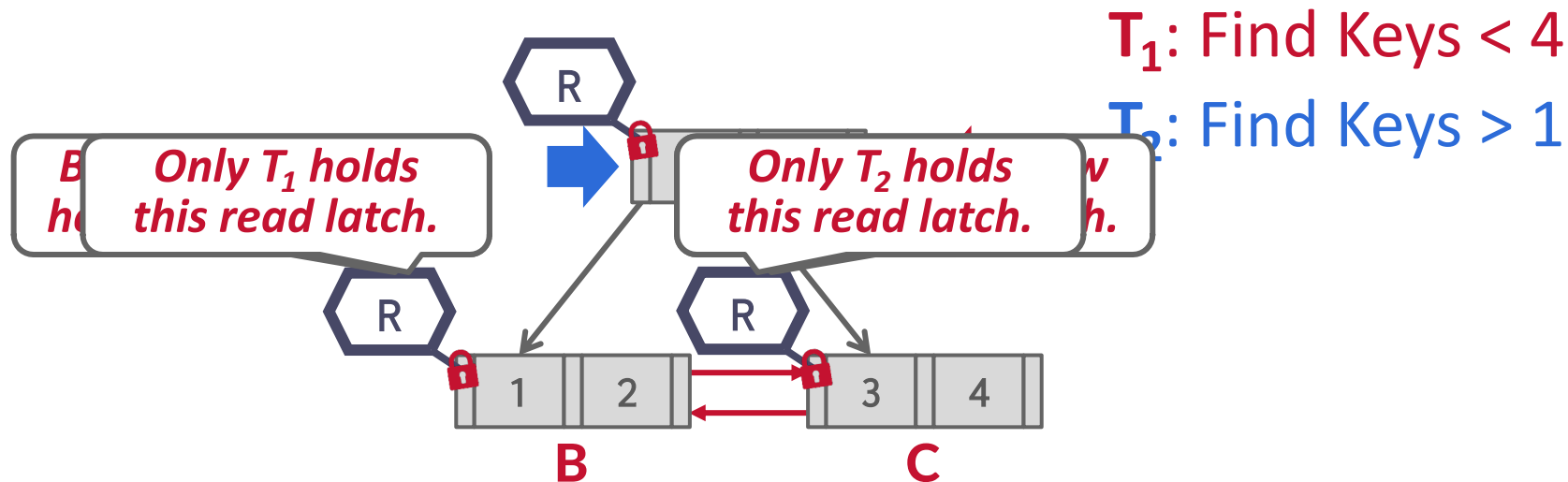
- A thread can only acquire a latch from a node that is below its current node.
- If the desired latch is unavailable, the thread must wait until it becomes available.

But what if threads want to move from one leaf node to another leaf node?

Leaf Node Scan Example #1



Leaf Node Scan Example #2



Leaf Node Scan Example #3

T₂ Choices?



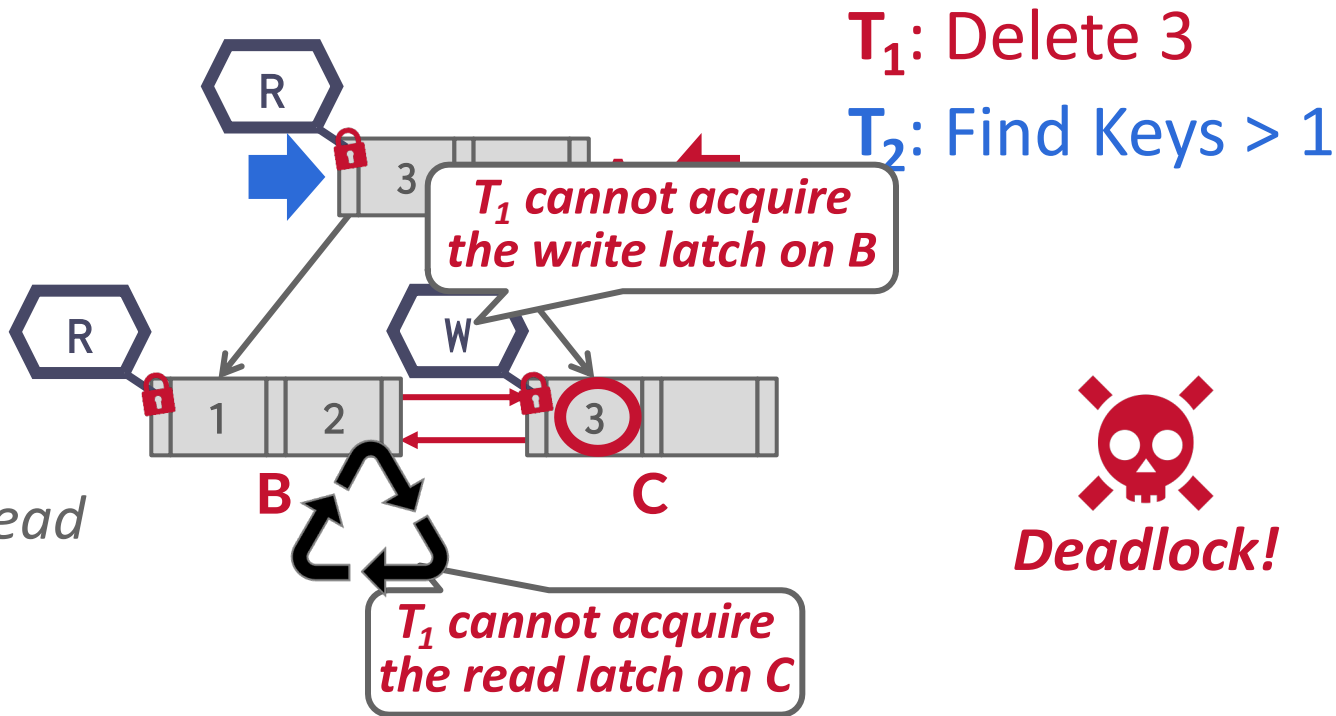
Wait



Give up



Kill Other Thread



Leaf Node Scans

Latches do not support deadlock detection or avoidance. The only way we can deal with this problem is through coding discipline.

The leaf node sibling latch acquisition protocol must support a “no-wait” mode.

The DBMS's data structures must cope with failed latch acquisitions.

→ Usually transparent to end-user / application.

Conclusion

Making a data structure thread-safe is notoriously difficult in practice.

We focused on B+Trees, but the same high-level techniques are applicable to other data structures.

Next Class

Hash tables and sorting in the real world



Project #2

You will build a thread-safe B+tree backed by your buffer pool manager.

- Page Layout
- Insert/Delete/Find Operations
- Iterator
- Latch Crabbing

We define the API for you. You need to provide the method implementations.

WARNING:

**This is more difficult than Project #1.
Start immediately!**

Tasks

Task #1: Page Layouts

- How each node will store its key/values in a page.
- You only need to support unique keys.

Task #2: Operations

- Support point queries (single key).
- Support inserts with node splitting.
- Support removal of keys with sibling stealing + merging.
- Does not need to be thread-safe.

Task #3: Index Iterator

- Create a STL iterator for range scans on leaf nodes.
- You only need to support ascending scans.

Task #4: Concurrent Index

- Introduce latch crabbing/coupling protocol to support safe concurrent operations.
- Make sure you have splits / merges working correctly before proceeding with this task.

Development Hints

Follow the textbook semantics and algorithms.

Set the page size to be small (e.g., 512B) when you first start so that you can see more splits/merges.

Make sure that you protect the internal B+Tree **root_page_id** member.

Extra Credit

Gradescope Leaderboard runs your code with a specialized in-memory version of BusTub.

The top 20 fastest implementations in the class will receive extra credit for this assignment.

- **#1**: 50% bonus points
- **#2–5**: 25% bonus points
- **#6–10**: 10% bonus points

You must pass all the test cases to qualify!